

Lab 1 (b)- Encrypting/Decrypting files using both Symmetric and Asymmetric Encryption: Handout

Time: 4 hours

When we encrypt/decrypt we have to use a password, but really long complex passwords are stored in files called keys. are done using **keys**. As for the encryption itself, there are 2 types of encryption:

- **Symmetric encryption:** With this type of encryption we have a single key. This key is used to encrypt data and is also used to decrypt it.
- **Asymmetric encryption (aka Public-key cryptography):** With this type of cryptography, we have a pair of keys (aka key-pair) which are intrinsically linked to each other. These keys are commonly referred to as the **public key** and **private key**. The public key can only encrypt files, but can't decrypt them again (you might wonder how it's possible that a key can decrypt but not encryp, the answer is that there is lot of [very complex maths](#) involved). The decrypting is actually done by the key-pair's private key. Note that it is also possible for the private key to created encrypted files which can then be decrypted by only the key-pair's public key (or the private key itself).

Encrypting and Decrypting data using symmetric Encryption

In the example we'll walkthrough how to encrypt a file using a symmetric key. A symmetric key can be in the form of a password which you enter when prompted. First we create a test file that is going to encrypted

```
$ echo 'hello world' > secrets.txt
```

Now we encrypt the file:

```
$ openssl aes-256-cbc -e -in secrets.txt -out secrets.txt.enc
```

```
enter aes-256-cbc encryption password:
```

```
Verifying - enter aes-256-cbc encryption password:
```

Now we have the encrypted file, secrets.txt.enc:

```
$ ls -l
```

```
total 8
```

```
-rw-r--r--. 1 root root 12 Mar  9 17:50 secrets.txt
```

```
-rw-r--r--. 1 root root 32 Mar  9 17:50 secrets.txt.enc
```

Here we used the ‘aes-256-cbc’ symmetric encryption algorithm, there are quite a lot of other symmetric encryption algorithms available.

Now to decrypt, we use the same key (i.e. password):

```
$ openssl aes-256-cbc -d -in secrets.txt.enc -out secrets.txt.dec
```

```
enter aes-256-cbc decryption password:
```

```
$ cat secrets.txt.dec
```

```
hello world
```

You can also use a key file to encrypt/decrypt:
first create a key-file:

```
$ openssl rand 128 > symmetric_keyfile.key

$ ll

total 8

-rw-r--r--. 1 root root  12 Mar  9 18:45 secrets.txt

-rw-r--r--. 1 root root 128 Mar  9 18:46 symmetric_keyfile.key
```

Now we encrypt like this:

```
$ openssl enc -in secrets.txt -out secrets.txt.enc -e -aes256 -k
symmetric_keyfile.key
```

This creates:

```
$ ll | grep secrets.txt.enc

-rw-r--r--. 1 root root  32 Mar  9 18:48 secrets.txt.enc
```

Now to decrypt it we do:

```
$ openssl enc -in secrets.txt.enc -out secrets.txt.decrypted -d -  
aes256 -k symmetric_keyfile.key
```

```
$ cat secrets.txt.decrypted
```

```
hello world
```

Encrypting and Decrypting data using Asymmetric encryption

```
$ openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt  
rsa_keygen_bits:2048
```

```
.....+++
```

```
.....+++
```

There are a lot of Asymmetric based Encryption Algorithms available. Here we specified the 'RSA' Asymmetric Encryption Algorithm which is the industry standard. Openssl initially generates a random number which it then uses to generate the private key. The longer this random number, the more complex the private key is which in turn makes the private key harder to crack using brute force. However a more complex private key also uses up more computing resources encrypting/decrypting data, that's why a balance needs to be struck. In my case I set the complexity to 2048 bits.

This command ends up creating the following file:

```
$ ll

total 4

-rw-r--r--. 1 root root 1704 Mar  8 13:32 private_key.pem

$ cat private_key.pem

-----BEGIN PRIVATE KEY-----

MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBAKggSkAgEAAoIBAQC4Du/lM5d8/jym
DaJud6oi5fbZcro3u7q7wa2JNbTXBtCbamCAbsBdss3jkbSN0shjRdK1X7GWfgsv
tbOIHHjh2WEUOT02q4ZPbwNYMk4ZvAzQBmr4gUfvJj6iyaYACdjebjzgsqQyVPcA
Ct2Aijue7OMms34wj61bCoGiRTG5h8NsoNRpU5ayqmrV0LFdK+QoA0/L163RbCMH
4t+ODgTkL39Q1t2DdZyWTIj1XX8J5R1U1Y2O5+851eurhYPAEPAA6kysgRbIYfEj
GpJtuV5NWVdCDtJ3h8pp6MSWy/Q6Aq/gulrlxWprgttJi/gvDRb2HvJzrEH2c6q2
+vuakdCzAgMBAAECggEACrSzt308voR7Op1oaUyElOsTZKqLslo1Dl7BnU2Ga0t2
LtQNMou3XLvdxfLfsbAk/8E79FjQl5RVSj0jaDNfxKNERkzeBqQLd+tZInQqnsyV
YjQ9XJ8IdByhYA6inTX1a1q2J8u8SUEv7My0/ZkMxr24qLogD2sMB1OiHrIvpj7q
DfNRtVlpqYBzoIT5NhPLx1OsbgXE90jclkoBn3GZjXBJVrgnLTGZi32InAb84F2W
6NlfspNiytKgQS1jkK12X5d8LcQhYV7p9eMj9eyM9HMoOCLD+YvImW2hRdj9Pwfu
YNFygkNLF4tMRrPsyRtie7MSWKWFtU2tPZ5ZaObXgQKBgQDnBG2c97+xlDgBN17z
```

```
4+pdAbo+gDMjXUltX6fSwVHn/e0BegvuZr9tsZm7Z4kLBgleQ6se08ByPXpRmIU0
McTXOpdC+1QriQ/270b+IQ48zlefFQktdEAB2OnuE2v9/759jQ3CMjDddyUXpw3S
1h3pzSSSBDGAeiUVz3kPsg+mYwKBgQDL9nqxn5JZgBJFBkmZReGBGOl3WCUUm9G
xZr0hq0AwiPzS0tpPwsnTHMhzC2TfNfEAZXH+X6zGYrxHz9jS5vxN0p1DuerwD2b
UunASFYNsDmaAMRcdolz3jW5iWa9AlAwqi/szPqJil4LffI59lZbKBFXUjVKlP6h
CjAOV7PVcQKBgCHAdN9Vfw20Dc4HipHpf6k9Al/1TQNissKnEhbppbTWVZU8ppeb
LWayZvqJYu1YYXADRPHy2ugV8VQqi42Xo+GqLsVbgO2nZcy3ItEoNTNPFmAoq6GX
dW6PqYXeEo4Wmlcvj44K1TbWHHY7MvrVy6MIFMq8kBrSPZTVLiPL6RtvAoGBALwB
tVMpngN19GE/zHPGYHlZcmQczbuqyPvZhl5EDhCW5ACNtTOMc0GETNQomxJKj7ET
LPeqWUnKD7L3dLyYo2QT7UQ9uvssNx2HrgQaKYgQt1oOvvHkKgtuFuOxbcxn53cU
SRzktUSfENVTAQbAtRVNt75f/tO9xgC3nVh7Kz9hAoGBANYkgv1AWGMrXON18xJX
jTfAlSsIlncg4KJXH+QVpNSdcBuI+duUg+VcTAL98HxR5Daah/iYpgdZZFUP2res
uwfxXc6VetTPaLwc9ZRrXzE+AyaDV4GosD9TZFzM5HB9gIqz9g+K28XUGtR85mA7
b8SkGyGB+PBb7JDZzxs/Kv0e
-----END PRIVATE KEY-----
```

We then generate this private key's public key by running the following:

```
$ openssl rsa -pubout -in private_key.pem -out public_key.pem
```

```
writing RSA key
```

Notice that the public key is generated using the private key as its input.

This command ended up creating the public key:

```
$ ll

total 8

-rw-r--r--. 1 root root 1704 Mar  8 13:32 private_key.pem

-rw-r--r--. 1 root root  451 Mar  8 13:34 public_key.pem

$ cat public_key.pem
-----BEGIN PUBLIC KEY-----
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAuA7v5TOXfP48pg2ibneq
IuX22XK6N7u6u8GtiTW01wbQm2pggHbAXbLN45G0jdLIY0XStV+xln4LL7WziBx4
4dlhFDk9NquGT28DWDJOGbwM0AZq+IFH7yY+osmmAAnY3m484LKkMlT3AArdgIo7
nuzjJrN+MI+tWwqBokUxuYfDbKDUaVOWsqpq1dCxXSvkKANPy5et0WwjB+Lfjg4E
5C9/UJbdg3WclkyI9V1/CeUdVJWNjufvOdXrq4WDwBDwAOpMrIEWyGHxIxqSbble
TVlXQg7Sd4fKaejElsv0OgKv4Lta5cVqa4LbSYv4Lw0W9h7yc6xB9nOqtv7mpHQ
swIDAQAB
-----END PUBLIC KEY-----
```

Now let's create some dummy data:

```
$ echo 'hello world' > PlainTextFile.txt

$ cat PlainTextFile.txt

H
```

```
ello world
```

Let's now create an encrypted version of this file, called EncryptedData.txt:

```
$ openssl rsautl -encrypt -inkey public_key.pem -pubin -in  
PlainTextFile.txt -out EncryptedData.encrypted
```

This ends up creating the following file:

```
$ ll  
  
total 16  
  
-rw-r--r--. 1 root root 256 Mar  8 13:46 EncryptedData.encrypted  
-rw-r--r--. 1 root root  12 Mar  8 13:42 PlainTextFile.txt  
-rw-r--r--. 1 root root 1704 Mar  8 13:32 private_key.pem  
-rw-r--r--. 1 root root  451 Mar  8 13:34 public_key.pem
```

The content of this file isn't readable since it is encrypted. So trying to cat this will just output gibberish instead of 'hello world'.

The EncryptedData.encrypted can't be decrypted by the public key, in fact the only file in the world that's capable of decrypting this file is the private key, private_key.pem. So to decrypt EncryptedData.encrypted, we need to run the following command:

```
$ openssl rsautl -decrypt -inkey private_key.pem -in  
EncryptedData.encrypted -out DecryptedData.txt
```

This command ends up creating the following file:


```
$ 11
```

```
total 20
```

```
-rw-r--r--. 1 root root  12 Mar  8 13:59 DecryptedData.txt
-rw-r--r--. 1 root root 256 Mar  8 13:46 EncryptedData.encrypted
-rw-r--r--. 1 root root  12 Mar  8 13:42 PlainTextFile.txt
-rw-r--r--. 1 root root 1704 Mar  8 13:32 private_key.pem
-rw-r--r--. 1 root root  451 Mar  8 13:34 public_key.pem
```

```
$ cat DecryptedData.txt
```

```
hello world
```

Success!

Encrypting with the Private Key and Decrypting with the Public Key

As mentioned earlier, with a public-private key pair, the role of the public key is to encrypt data so that it can only be decrypted by the private key, and the role of the private key is to only be able to decrypt data that has been encrypted by its public key counterpart.

However due to the nature of Asymmetric cryptography, it is possible to do the reverse. I.e. the private key can encrypt data that can be decrypted by the key-pair's public.

Let's say we have both private and public keys from earlier on, as well as the sample plain text data:

```
$ 11
```

```
total 12
```

```
-rw-r--r--. 1 root root  15 Mar  8 18:27 PlainTextFile.txt

-rw-r--r--. 1 root root 1704 Mar  8 14:21 private_key.pem

-rw-r--r--. 1 root root  451 Mar  8 14:29 public_key.pem

$ cat PlainTextFile.txt

hello world!!!
```

Now to encrypt the plaintext data with the private key, we run the following command:

```
$ cat PlainTextFile.txt | openssl rsautl -inkey private_key.pem -
sign > data_thats_encrypted_with_private_key.encrypted
```

If you cat this output file then it will just display gibberish since it's encrypted.

So to decrypt this file we use the public key like this:

```
$ openssl rsautl -inkey public_key.pem -pubin -in
data_thats_encrypted_with_private_key.encrypted

hello world!!!
```

This ability of encrypting data using the private key is actually used in real-life, as part of creating a Certificate Signing Request (CSR) file and getting it signed by a Certificate Authority. During a CSR's creation a

digital signature is attached. The digital signature is essentially the public key's hash encrypted by the private key:

```
$ md5sum public_key.pem | openssl rsautl -inkey private_key.pem -  
sign > checksum.signed
```

The CA uses this digital signature (which we called checksum.signed in this example) to validate that the creator of the CSR also holds the private key. It does this by attempting to decrypt the signature using the public key which is also attached to the CSR in unencrypted form. I.e. the CA runs the following:

```
$ openssl rsautl -inkey public_key.pem -pubin -in checksum.signed  
  
f0be71a5a8e3cb907b4b4c2f2fc473b4  public_key.pem
```

The fact that the CA managed to decrypt the signature proves that the creator of the CSR also holds the private key. Since the public key wouldn't have been able to decrypt the file unless it was encrypted by the private key. As an extra check the CA will compare this output with the public key's hash:

```
$ md5sum public_key.pem  
  
f0be71a5a8e3cb907b4b4c2f2fc473b4  public_key.pem
```

The 2 should match (which in this example it does), then it proves to the CA.